

PROMPT IA 12

Search & Discovery avance - NOMA

Search Service + OpenSearch + Kafka + autocomplete + facettes + ranking + analytics

Mission : industrialiser la recherche et la decouverte produit du Storefront NOMA sans casser les responsabilites Product, Catalog, Pricing, Inventory et Review.

Source de verite metier: PostgreSQL - Projection Search reconstruisible: OpenSearch

1. Role de l'IA

Tu es Principal Search Engineer et Backend Go Engineer. Implemente le Search Service et ses projections OpenSearch dans le monorepo existant. Reutilise les conventions Go, Kafka, Protobuf, observabilite, securite et GitOps deja etablies. Ne duplique aucune logique metier appartenant aux domaines sources.

Search n'est jamais la source de verite. Un index OpenSearch doit pouvoir etre supprime puis reconstruit depuis les sources durables et les evenements.

2. Sources visuelles et UX

Le Design Pack Storefront NOMA et le Prompt IA 10 sont les references visuelles et frontend. Ne cree pas une nouvelle identite Search. Les composants Search, autocomplete, filtres, facettes, tri, no-results, loading et mobile filter drawer reutilisent les tokens et composants NOMA existants.

Le Prompt 12 implemente principalement le moteur, les contrats et l'integration. Les changements UI doivent rester compatibles avec Next.js + TypeScript + Tailwind, mobile-first, responsive Flexbox/Grid, RSC/SSR et budget JavaScript mesure.

3. Ownership des donnees

Domaine	Donnees proprietaires	Projection Search
Product	titre, SKU, attributs, medias, base price	document produit
Catalog	categories, collections, merchandising, visibilite	navigation/facettes
Pricing	prix effectif, promotions	prix filtre/tri affiche
Inventory	disponibilite/stock publie	availability
Review	note moyenne, nombre avis	rating/review_count
Search	index, analyzers, ranking, suggestions	projection reconstruisible

4. Architecture cible

```
Product ----\
Catalog -----\
Pricing -----+--> Kafka --> Search Projector --> OpenSearch
Inventory ----/ |
Review -----/ +--> aliases/versioned indices
|
Storefront Next.js --> Search API (REST) --> Search Service --> OpenSearch
```

Les mises a jour de projection sont asynchrones via Kafka. L'API publique Search est REST/JSON. Les appels internes synchrones eventuels suivent les conventions gRPC existantes, mais aucun appel fan-out vers cinq services n'est autorise sur chaque requete de recherche.

5. Contrat Search API

```
GET /v1/search?q=running+shoe&category;=men&brand;=noma
&price;_min=50&price;_max=200&_stock=true
&sort;=relevance&cursor;=...&limit;=24

GET /v1/search/suggest?q=run&limit;=8
GET /v1/search/facets?...
GET /v1/search/health
```

Utiliser pagination par curseur/search_after pour les parcours profonds. Limiter strictement limit, longueur de query, nombre de filtres et complexite. Valider les enums et champs filtrables cote serveur.

6. Document OpenSearch

```
product_id
sku_ids[]
title
normalized_title
description_search
category_ids[]
collection_ids[]
brand
attributes{}
price_effective
price_currency
promotion_flags[]
in_stock
availability_status
rating_avg
review_count
visibility
merchandising_score
popularity_score
updated_at
source_versions{}
```

Ne pas indexer de PII. Ne pas recopier de champs inutiles. Chaque champ doit avoir une justification de recherche, filtre, facette, tri ou affichage.

7. Mapping et analyzers

Définir explicitement mappings et analyzers. Interdire le mapping dynamique non contrôlé en production. Utiliser keyword pour filtres/agrégations, types numériques pour prix/rating, dates pour timestamps et text avec sous-champs adaptés pour full-text.

Prévoir normalisation accents/casse, tokenisation adaptée aux langues supportées, synonymes versionnés et edge-ngram/search-as-you-type uniquement là où cela apporte un gain mesuré.

8. Full-text et pertinence

La pertinence doit être explicable et testable. Commencer simple avec BM25 puis ajouter boosts métier mesurés.

```
score = text_relevance
+ title_exact_boost
+ category_context_boost
+ merchandising_boost
+ popularity_signal
+ availability_signal
+ optional_review_signal
```

Ne jamais laisser le ranking commercial rendre invisibles les résultats textuellement pertinents sans règle documentée. Toute modification de ranking doit être versionnée et testée offline/online.

9. Autocomplete et suggestions

Autocomplete doit répondre rapidement et supporter annulation/debounce côté frontend. Fournir suggestions produits, catégories et requêtes seulement si elles ont une valeur mesurée.

```
user types: run
-> Running shoes
-> Running jacket
-> Category: Running
-> Product: NOMA Run Pro
```

Limiter la fuite de données : ne jamais générer de suggestion depuis des recherches personnelles identifiables. Les analytics de requêtes sont agrégés/anonymisés selon la politique privacy.

10. Typo tolerance, synonymes et zero-result

Implementer une tolerance aux fautes bornees. Eviter les fuzzy queries couteuses partout. Les synonymes sont versionnes, testables et deployables sans modification manuelle de production.

Pour zero-result : retourner suggestions orthographiques, categories proches ou suppression controlee de filtres. Ne jamais inventer de produits.

11. Facettes et filtres

Supporter categories, marque, attributs pertinents, prix, disponibilite, note et autres facettes justifiees. Les facettes doivent respecter le contexte des filtres actifs et rester coherentes avec le resultat.

Eviter les aggregations non bornees et les cardinalites explosives. Ajouter des limites et timeouts.

12. Tri

```
relevance
price_asc
price_desc
newest
popularity
rating
```

Chaque tri doit etre stable et avoir un tie-breaker deterministe. Le tri par prix utilise le prix effectif projete fourni par Pricing, jamais un recalcul local Search.

13. Indexation evenementielle

Consommer les evenements versionnes des domaines sources avec idempotence, retries, DLQ, correlation_id et trace_id. Utiliser aggregate_id comme cle de partition lorsque la convention du domaine l'impose.

```
product.updated.vN
catalog.visibility_changed.vN
pricing.effective_price_changed.vN
inventory.availability_changed.vN
review.aggregate_changed.vN
|
v
Search Projector -> partial/full document update
```

Conserver source_versions afin d'ignorer les evenements anciens/out-of-order quand necessaire.

14. Reindexation sans downtime

```
products-v41 <- current alias: products-read
|
rebuild products-v42
|
validation + document counts + sample queries
|
atomic alias switch
v
products-v42 <- products-read
|
rollback alias possible
```

Automatiser create -> backfill -> validate -> alias swap -> observe -> retire old index. Ne jamais reconstruire directement l'index actif.

15. Reconstruction et reconciliation

Fournir une commande/job de rebuild complet et un mecanisme de reconciliation entre sources durables et projection. Detecter documents manquants, obsoletes ou orphelins.

Le rebuild doit etre throttle pour ne pas saturer PostgreSQL, Kafka ou OpenSearch.

16. Search analytics

Collecter uniquement des signaux utiles et minimises : query normalisee, result_count, latency, selected_position, no_result, conversion attribution si autorisee. Preferer des evenements pseudonymises/agreges.

```
search.performed.v1
search.result_clicked.v1
search.zero_result.v1
```

Ces evenements peuvent alimenter Kafka -> ClickHouse/dbt pour analyser zero-results, CTR et qualite du ranking. Ils ne modifient pas directement le ranking en production sans pipeline valide.

17. Performance et SLO

Objectif existant : recherche Catalog/Product p95 < 250 ms. Definir aussi budgets internes pour OpenSearch query, Search Service, autocomplete et projection lag.

```
Search API p95 < 250 ms
Autocomplete target p95 < 150 ms
OpenSearch timeout borne
Projection lag observable
Error rate SLO-backed
```

Executer des tests k6 avec requetes realistes, filtres, facettes, autocomplete et cache chaud/froid. Mesurer p50/p95/p99 et saturation.

18. Cache

Ajouter du cache seulement apres mesure. Les requetes populaires et donnees stables peuvent utiliser Redis/HTTP cache selon semantics. Les resultats sensibles au prix/stock doivent respecter une freshness explicite.

Ne jamais utiliser le cache pour masquer une projection incorrecte.

19. Securite et anti-abus

Search est public mais protege : Kong rate limiting, limites de query, timeouts, validation stricte et refus des DSL OpenSearch arbitraires venant du client. Aucun utilisateur ne peut envoyer directement une query DSL au cluster.

OpenSearch reste prive dans Kubernetes. Credentials via OpenBao, NetworkPolicies Cilium default-deny, trafic mesh selon Istio Ambient.

20. Observabilite

```
search_requests_total
search_latency_seconds
search_zero_results_total
search_result_count
search_opensearch_latency_seconds
search_opensearch_errors_total
search_projection_lag_seconds
search_projection_dlq_total
search_reindex_duration_seconds
search_reindex_validation_failures_total
```

Tracer Search API -> OpenSearch et la chaine evenement -> projector. Logs structures sans PII. Dashboards Grafana et alertes sur SLO, erreurs, lag, DLQ et saturation.

21. Tests de pertinence

Creer un corpus de golden queries versionne dans Git avec resultats attendus/acceptables. Les changements analyzers, boosts ou synonymes doivent passer ces tests avant merge.

```
query: "running shoes"
expected_top_k:
- category=running
- in_stock=true preferred
forbidden:
- hidden products
- archived products
```

Ajouter NDCG/MRR ou metriques equivalentes lorsque le corpus devient suffisamment riche. Les seuils doivent etre documentes et comparables entre versions.

22. Tests obligatoires

Scenario	Attendu
Product event duplicate	Projection idempotente.
Out-of-order event	Version ancienne ignoree ou reconciliee.
Pricing update	Prix filtre/tri actualise sans logique Pricing locale.
Inventory update	Disponibilite projete.
Hidden product	Jamais retourne publiquement.
Typo	Correction/tolerance bornee.
Zero result	Fallback controle, aucun produit invente.
OpenSearch timeout	Erreur/fallback propre et observable.
Reindex	Alias swap sans downtime.
Rebuild	Index reconstruisible.
Load test	p95 cible respecte sous charge validee.

23. Integration Storefront

Integrer le Search Service au Storefront du Prompt IA 10. Preferer RSC/SSR pour le rendu initial des pages Search/Catalogue et fetch client cible pour autocomplete ou interactions necessitant une reponse immediate.

Initial search/category:
Browser -> ATS -> Next.js RSC/SSR -> Search API -> OpenSearch

Autocomplete:
Browser -> debounce + AbortController -> Search API -> OpenSearch

Preserver URL partageable pour query/filtres/tri, navigation back/forward, SEO des pages indexables et accessibilite clavier de l'autocomplete.

24. Pelias + libpostal: perimetre separe

Pelias/libpostal concernent l'adresse checkout et non la recherche produit. Ne pas les brancher dans OpenSearch Product Search. Ils peuvent etre integres via un adapter Address/Search dedie utilise par le checkout du Prompt 10.

Product discovery -> Search Service -> OpenSearch
Address autocomplete -> Address adapter -> Pelias/libpostal

25. CI/CD et GitOps

Versionner mappings, templates, analyzers, synonymes, golden queries et jobs de reindexation. CI : gofmt/vet/lint -> unit/integration -> contract -> relevance tests -> k6 smoke -> security scan -> OCI -> SBOM -> Cosign -> Harbor -> FluxCD.

Les changements incompatibles de mapping doivent creer un nouvel index versionne, jamais modifier dangereusement l'index actif.

26. Ordre d implementation

1. Search domain/API contracts
2. OpenSearch index template + mappings

3. Product/Catalog projection
4. Pricing/Inventory/Review projection
5. Search API full-text
6. Filters/facets/sorts
7. Autocomplete/typo/synonyms
8. Reindex + alias workflow
9. Reconciliation/rebuild
10. Search analytics
11. Storefront integration
12. Relevance + load + failure tests
13. Observability
14. GitOps/runbooks

27. Definition of Done

- Recherche full-text fonctionnelle avec filtres, facettes et tris.
- Autocomplete rapide, accessible et annule les requetes obsoletes.
- Product/Catalog/Pricing/Inventory/Review alimentent une projection coherente.
- Aucune logique metier Pricing/Inventory n'est dupliquee dans Search.
- Produits caches/archives ne fuient pas dans les resultats publics.
- Reindexation versionnee avec alias et rollback fonctionne sans downtime.
- Index completement reconstruisible depuis les sources durables.
- Golden queries et tests de pertinence passent.
- p95 Search respecte l'objectif < 250 ms sous charge cible.
- Metrics, traces, logs, lag et DLQ sont observables.
- Storefront NOMA reutilise son design existant pour Search/Autocomplete/Filters.
- Tout est deployable via CI/CD + FluxCD.

Ne generalise pas prematurement. Livre d'abord un vertical slice Product + Catalog + Pricing + Inventory -> Kafka -> OpenSearch -> Search API -> Storefront, mesure sa pertinence et sa latence, puis ajoute Review, analytics et optimisations.

28. Livrables attendus de l'IA

Produire le code Go, contrats API, consumers Kafka, mappings/templates OpenSearch, migrations/configurations necessaires, tests, golden-query corpus, scripts/jobs de rebuild/reindex, dashboards/alertes, manifests GitOps, documentation d'exploitation et runbooks de panne/reindexation.

A la fin, fournir un rapport court : fichiers modifies, decisions prises, tests executes, resultats de performance, risques restants et prochaines optimisations mesurees.